



Flask Reels Web Application

An Instagram-inspired short video upload and viewer platform built with **Python (Flask)** — lightweight, file-based, and ready to extend.

PRESENTED BY VEDANT BAWASKAR

2ND YEAR CSE

PRESENTED BY ANJALI WAKALKAR

2ND YEAR CSE

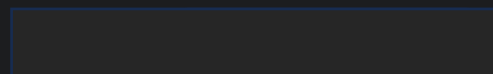
PRESENTED BY SUHANI DESHMUKH

2ND YEAR CSE

Introduction

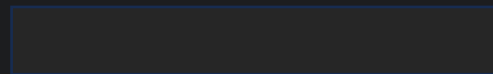
Flask Reels is a **full-stack mini web application** that replicates the core experience of short-form video platforms like Instagram Reels. Built entirely with Python and Flask, it lets users upload, browse, and manage short video content — all without a traditional database setup.

The project focuses on practical learning: real file handling, metadata management with JSON, secure uploads, and a clean responsive UI built with plain HTML and CSS.



What Users Can Do

- Upload short video reels with thumbnails
- Browse content in reverse chronological order
- Delete reels and associated media files
- Experience secure, validated file handling



Project Objectives

This project was designed as a hands-on exploration of backend web development fundamentals, spanning framework usage, file I/O, data persistence, and security.

1

Build a Full-Stack App

Design and implement a working web application with both frontend templates and backend logic using Flask.

2

Master Flask Framework

Understand routing, request handling, template rendering, and static file serving in a real project context.

3

Handle File Uploads

Implement secure multi-file upload with MIME validation, size limits, and UUID-based unique naming.

4

Apply Security Practices

Use `secure_filename`, restrict allowed extensions, and prevent directory traversal vulnerabilities.

Tech Stack

Python 3

Core backend language powering all application logic and file operations.

Flask

Lightweight WSGI web framework handling routing, requests, and template rendering.

HTML / CSS

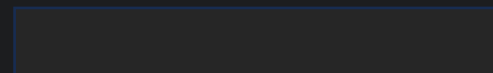
Jinja2 templates for dynamic frontend rendering with custom stylesheets.

JSON

Flat-file metadata storage — no database setup required, easy to inspect and debug.

UUID

Generates unique filenames on upload to prevent conflicts and overwriting.



System Architecture

The application follows a straightforward request-response cycle. The Flask server acts as the central hub, coordinating file storage and metadata management before serving content back to the browser.



User Upload

Flask Server

File Storage

JSON Metadata

All uploaded videos and thumbnails are stored in the `static/` directory, while reel metadata (title, filename, timestamp) is persisted in `data/reels.json` — keeping the stack simple and self-contained.

Project Workflow

Every upload follows a strict, predictable sequence — from user action to visible result on the homepage.

1

User Uploads

Selects a video file (MP4/WebM) and an optional thumbnail (JPG/PNG/WEBP) via the upload form.

2

Validation

Flask checks MIME type and file extension against an allowed list; oversized or invalid files are rejected.

3

UUID Naming

A unique identifier is generated and prepended to the filename to prevent collisions in the static folder.

4

File Storage

Video saved to `static/reels/`, thumbnail to `static/thumbnails/`.

5

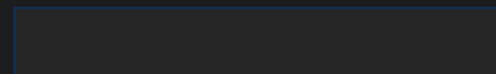
Metadata Saved

Title, filenames, and upload timestamp are appended to `data/reels.json`.

6

Display

Homepage reloads and renders all reels in reverse chronological order using Jinja2 templates.



Project Structure

Directory Layout

```
project/
├── main.py           # Flask app & routes
├── requirements.txt  # pip dependencies
├── static/
│   ├── reels/       # Uploaded videos
│   ├── thumbnails/  # Uploaded thumbnails
│   └── songs/        # Optional audio
└── data/
    └── reels.json    # Metadata store
```

Key Files Explained

- `main.py` — defines all routes: upload, delete, index, and static serving
- `reels.json` — flat list of reel objects with title, filenames, and timestamp
- `requirements.txt` — only Flask needed; install with a single `pip` command
- `static/` — Flask serves this folder directly to the browser with no extra config

Security Features



Secure Filenames

Werkzeug's `secure_filename()` strips dangerous characters and path separators from user-supplied filenames before saving.



Extension Whitelist

Only explicitly allowed extensions (`.mp4`, `.webm`, `.jpg`, `.png`, `.webp`) are accepted; all others return an error.



80 MB Upload Cap

Flask's `MAX_CONTENT_LENGTH` is set to 80 MB, preventing server overload from excessively large files.



No Directory Traversal

Combining UUID prefixing with `secure_filename()` ensures uploads cannot escape the designated static folder.

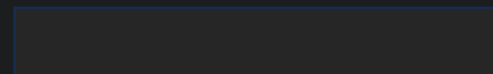
Advantages & Limitations

✓ Advantages

- Zero database setup — JSON storage means instant deployment with no SQL config.
- Lightweight stack — Flask's minimal footprint keeps the codebase readable for beginners.
- Great learning project — covers routing, file I/O, templates, and security in one app.

⚠ Limitations

- No authentication — any visitor can upload or delete content with no access control.
- JSON is not scalable — concurrent writes risk data corruption; unsuitable for production use.
- No cloud storage — files stored locally; no redundancy or CDN delivery for media.



Future Scope

Planned Improvements

01

User Authentication

Login/register system so only authorised users can upload or delete their own reels.

02

Database Integration

Replace JSON with SQLite or MySQL for reliable, concurrent, and scalable data storage.

03

Social Features

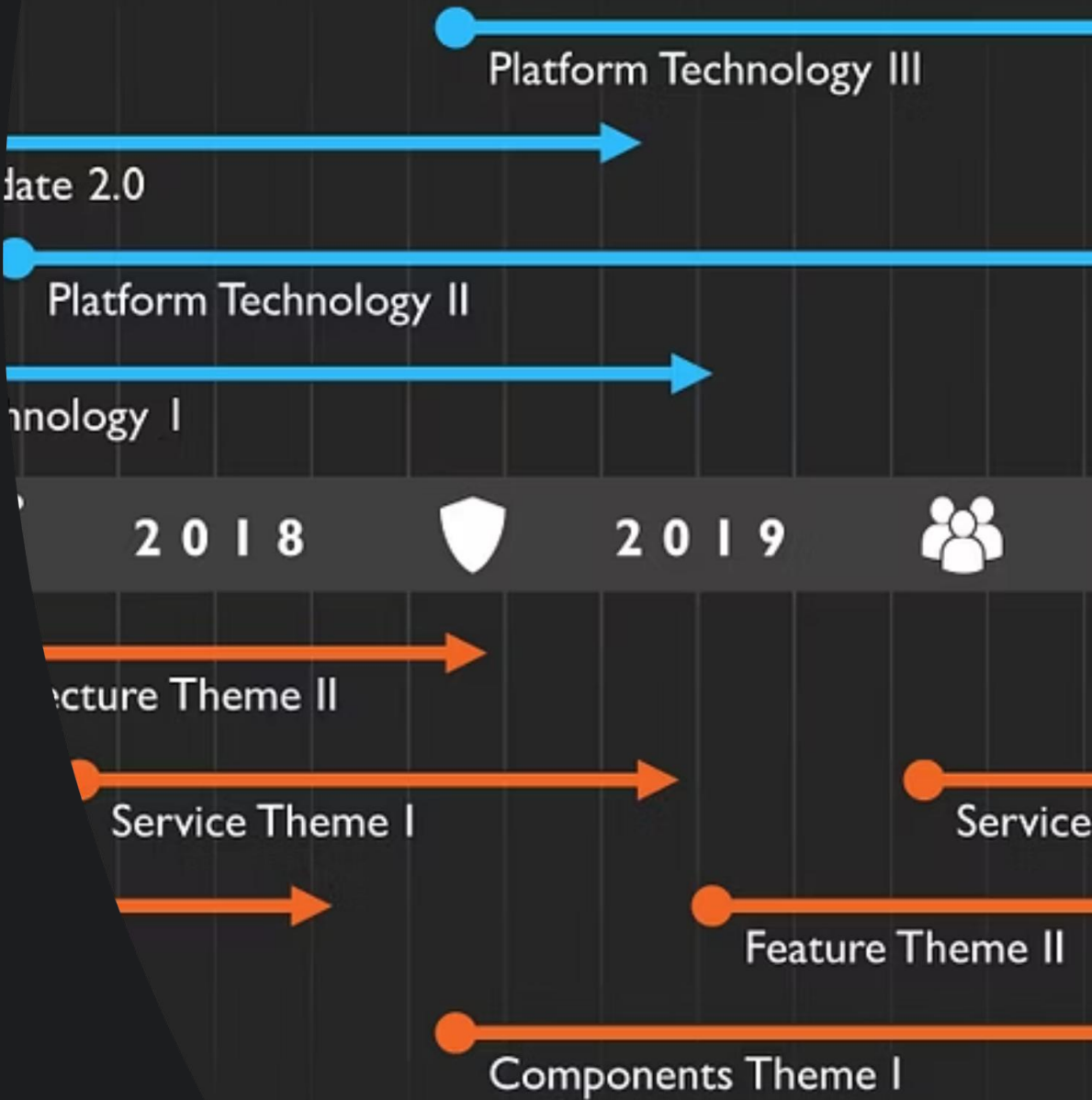
Like and comment system to enable community interaction around uploaded content.

04

Cloud Deployment

Host on AWS or Heroku with cloud storage (S3) for media — making the app publicly accessible.

TECHNOLOGY ROAD



CONCLUSION

- Successfully built a working reels platform with Flask
- Gained practical experience in file handling, routing, and backend logic
- Applied real-world security practices from day one
- Strong foundation to extend into a production-grade application.

This project successfully demonstrates the development of a functional reels platform using Flask, providing hands-on experience in backend development, file handling, and routing. It incorporates essential security practices such as safe file uploads and validation, ensuring reliable performance. Overall, the project builds a strong foundation for developing scalable and production-ready web applications in the future.

